



SPAREPARTS · IGNITION DESIGN PACKAGE

REPORT 03 OF 6

UI Component Architecture & Patterns

Design-token system, new Ignition component inventory, composition maps for the Dashboard and Client Workspace, responsive strategy, interaction engineering, performance budget, accessibility implementation, and a phased build plan — scoped to the existing no-build hyperscript React runtime in `SpareParts.Web.React/wwwroot`.



Author

dev-web-react — Web Frontend Agent

Date

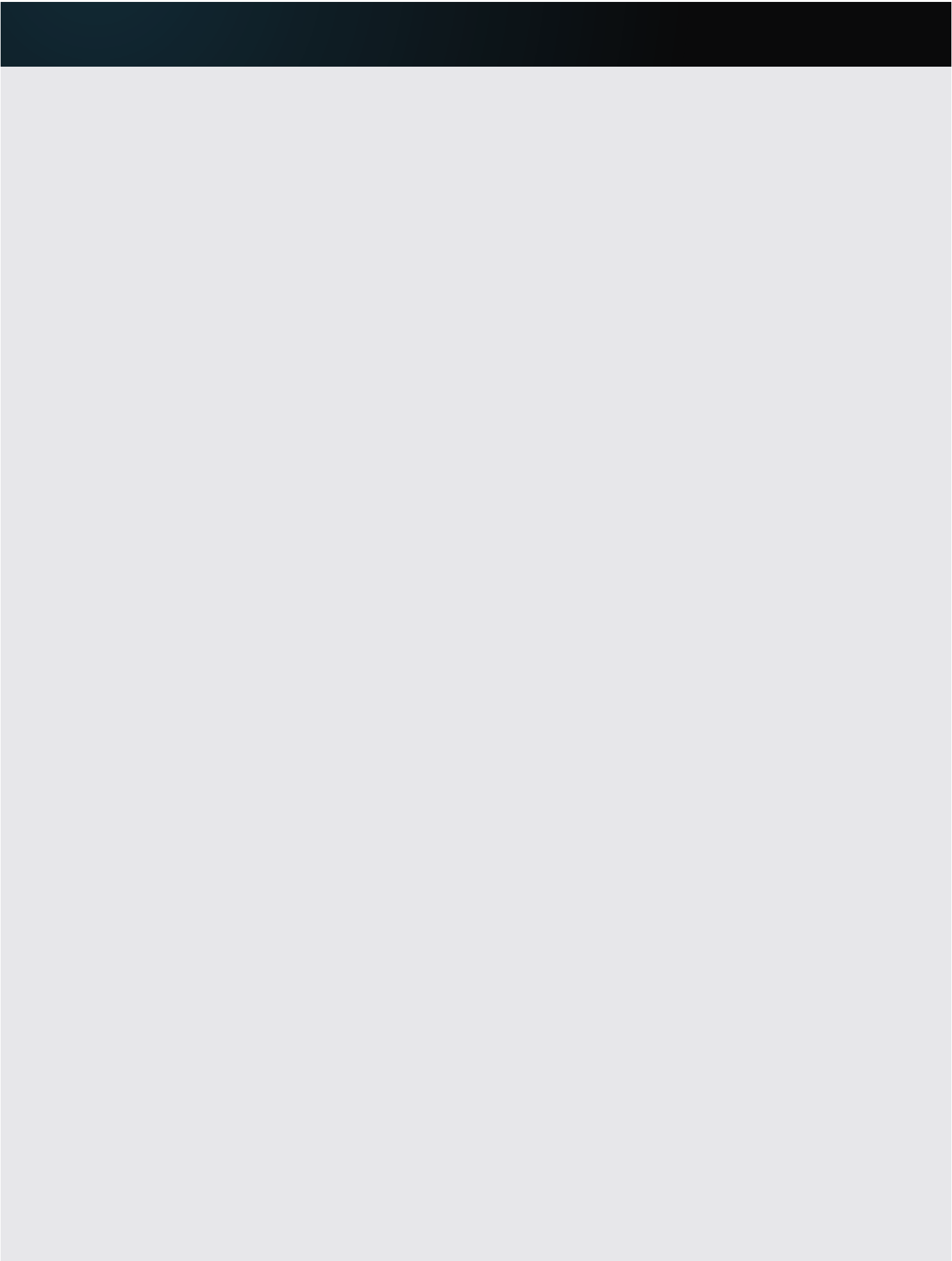
2 July 2026

Codename

Ignition

Scope

Dashboard + Client Workspace



Report 03 — Table of Contents

Nine sections covering the component architecture that implements the Ignition design language inside the current no-build React runtime, without breaking the other 50+ screens still running on the legacy styles.css.

01	Executive Summary	p.03
02	Design-Token Architecture	p.04
03	New Component Inventory — Shell & Data	p.05
04	New Component Inventory — Client Workspace	p.06
05	Component States & Specimen Renders	p.07
06	Composition Map — Dashboard Screen	p.08
07	Composition Map — Client Workspace Screen	p.09
08	Responsive Strategy	p.10
09	Interaction Engineering	p.11
10	Performance Budget	p.12
11	Accessibility Implementation	p.13
12	Implementation Plan & File Map	p.14

Figures: 6 inline SVG diagrams — token cascade, component-tree (Dashboard), component-tree (Client Workspace), responsive reflow, keyboard/focus flow, and a rendered KPI/AgingBar specimen sheet.

State of the Component Layer & Target Architecture

CURRENT STATE (VERIFIED IN CODEBASE)

The React web app is a genuine no-build runtime: `js/core/react-runtime.js` re-exports `window.React` / `window.ReactDOM` and aliases `React.createElement` to `h`. There is no JSX, no bundler, no npm build step — every view is an ES module imported directly by the browser and composed with `h(tag, props, ...children)` calls. `js/views/screen-registry.js` wires 70 screens into `ScreenRegistry`, and `js/components/ui/` already holds a first-generation kit: `AppShell`, `Sidebar`, `Topbar`, `DataTable`, `StatsCard`, `StatusPill`, `Drawer`, `Modal`, `LoadingSkeleton`, `EmptyState`, `PageHeader`, `SearchBar`, `ActionBar`, `FilterPanel`, `Badge`, `FormSection`, plus domain cards (`PartCard`, `SellerCard`, `VehicleCard`). Styling is one 17,400-line `styles.css` (plus a 392-line `css/cockpit.css` overlay already carrying the rejected "Cockpit/Aurora" navy-teal look with `ck-*` class names visible directly in `dashboard-view.js` and `Sidebar.js` / `Topbar.js` / `AppShell.js`). Client data today is fragmented: `contacts-view.js` renders two flat "Customers / Suppliers" balance lists, `customer-aging-view.js` is a standalone aging-bucket table with a "Send Reminder" action, and invoices/quotes/part-requests/loyalty each show customer context independently with no unified 360° view and no shared client-selection state.

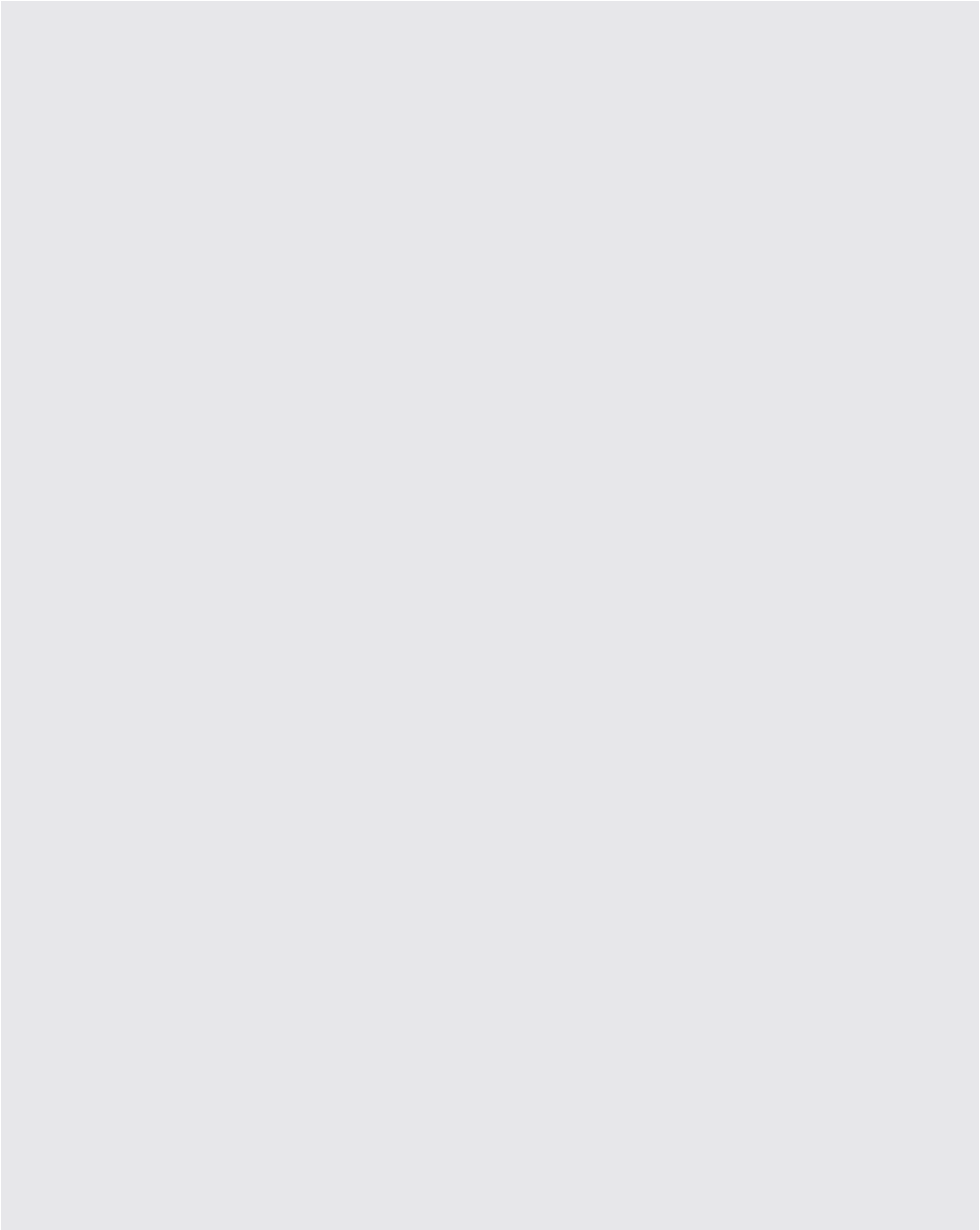
TARGET ARCHITECTURE

Ignition introduces a **four-layer CSS cascade** (tokens → primitives → components → views) that coexists with the legacy stylesheet during migration, a **second-generation component set** under a new `js/components/ignition/` namespace that composes over — never replaces in place — the existing UI kit, and two flagship screens: a rebuilt `dashboard-view.js` and a new `client-workspace-view.js` that unifies contacts, aging, invoices, payments, quotes, part requests, vehicles, loyalty, and communications into one customer-360 shell. Both consume the same `ApiClient` (`js/core/api-client.js`, unchanged) and the same `ScreenRegistry` wiring pattern already used by all 70 screens, so shipping Ignition on 2 screens is additive and reversible — the other 68 screens keep rendering exactly as they do today, untouched, on `styles.css`.

70 SCREENS IN REGISTRY TODAY	17.4k LINES IN STYLES.CSS	2 SCREENS SHIPPING IN V1	0 BUILD STEP REQUIRED
--	-------------------------------------	------------------------------------	---------------------------------

NON-NEGOTIABLE CONSTRAINTS THIS ARCHITECTURE HONORS

- No JSX, no transpile — every new file is a plain ES module using `h()`, importable by `<script type="module">` directly from `wwwroot`.
- One type/component per file — every Ignition primitive and composite gets its own file, named after the export, mirroring the existing `js/components/ui/*.js` convention.
- Zero regressions on the 68 untouched screens — token layer is additive CSS custom properties scoped under an opt-in root class, not a global reset.
- Existing `ApiClient` contract (`api.get/list/post/put/delete/postForm`, `ApiError` shape) is the only data-access surface — no new fetch layer.



Replacing 17,400 Lines with a Layered System

The token architecture is four physical files loaded in strict cascade order, added as new `<link>` tags *after* `styles.css` in `index.html` so Ignition rules win on selectors of equal specificity without deleting or rewriting the legacy file yet. Ignition surfaces are opt-in via a single root class, `.ignition-root`, applied only to the two new views' outermost container — the 68 legacy screens never receive this class and never see a token change.



TOKEN FILE (EXCERPT)

```
:root {
  /* Surfaces (dark-first) */
  --ig-carbon: #0a0a0b;
  --ig-surface: #131316;
  --ig-elevated: #1b1b1f;
  --ig-line: rgba(255,255,255,.08);

  /* Accent system */
  --ig-orange: #f97316;  --ig-amber: #fbbf24;
  --ig-success: #22c55e;  --ig-danger: #ef4444;  --ig-info: #38bdf8;

  /* Text */
  --ig-text-1: #fafafa;  --ig-text-2: #a1a1aa;

  /* Type */
  --ig-font-display: 'Plus Jakarta Sans', sans-serif;
  --ig-font-body: 'Inter', sans-serif;
  --ig-font-mono: 'JetBrains Mono', monospace;

  /* Shape / elevation */
  --ig-radius: 16px;  --ig-radius-pill: 999px;
  --ig-shadow-1: 0 1px 2px rgba(0,0,0,.4), 0 8px 24px -8px rgba(0,0,0,.5);
  --ig-shadow-2: 0 2px 4px rgba(0,0,0,.5), 0 16px 40px -12px rgba(0,0,0,.6);

  /* Motion */
  --ig-ease: cubic-bezier(.2,.8,.2,1);  --ig-dur-fast: 120ms;  --ig-dur: 200ms;
}

[data-ig-theme="light"] {
  --ig-carbon: #f4f4f6;  --ig-surface: #ffffff;  --ig-elevated: #ffffff;
  --ig-text-1: #131316;  --ig-text-2: #555555;  --ig-line: rgba(10,10,11,.08);
}
```

MIGRATION PATH (DOES NOT BREAK THE OTHER 50 SCREENS)

STEP	ACTION	BLAST RADIUS
1	Add <code>tokens.css</code> + <code>primitives.css</code> as new files, linked after <code>styles.css</code> . No selectors match existing markup (new <code>--ig-*</code> vars, <code>.ignition-root</code> scope).	Zero — additive only
2	Build <code>components.css</code> for the new inventory (Section 3–4). Every rule is prefixed <code>.ignition-root .ig-*</code> .	Zero

3	Ship <code>dashboard-view.js</code> v2 wrapped in <code><div class="ignition-root"></code> ; old <code>ck-*</code> rules in <code>cockpit.css</code> stay in the file but are simply unreferenced once the view stops emitting those class names.	1 screen
4	Ship <code>client-workspace-view.js</code> ; retire routing to <code>contacts-view.js</code> / <code>customer-aging-view.js</code> from the sidebar (files kept, just unlinked from <code>screen-registry.js</code>) pending Ralph's approval to delete.	2 screens
5	Phase 2 (future report): migrate remaining 68 screens screen-by-screen; only then prune <code>cockpit.css</code> and dead rules from <code>styles.css</code> .	Out of scope for this report

Shell, Command & Data Components

All components below live in `js/components/ignition/`, one file per export, same `h()` hyperscript pattern as the existing kit. Each accepts a plain-object `props` arg (no JSX spread sugar available) and is stateless where possible, lifting state to the view per existing conventions (`dashboard-view.js` , `invoices-view.js` keep state in the view and pass callbacks down).

ICONRAIL

PROP	TYPE	NOTES
<code>items</code>	<code>Array<{key,icon,label,badge?}></code>	Pulled from <code>screenRegistry</code> curated groups, same source as current <code>Sidebar.js railItems</code>
<code>activeKey</code>	string	Current view key from router state
<code>onSelect(key)</code>	fn	Delegates to <code>onView</code> callback already threaded through every view
<code>flyout</code>	bool	72px collapsed rail; hover/focus reveals a 200px label flyout via CSS <code>:hover</code> / <code>:focus-within</code> , no JS state needed

States: default (icon only, 40% opacity label) · hover/focus (flyout slides in 120ms, full opacity) · active (orange left rail indicator + filled icon bg) · badge (unread dot, reuses `StatusPill` tone tokens).

COMMANDBAR + COMMANDPALETTE

PROP	TYPE	NOTES
<code>onSearch(query)</code>	fn	Debounce 180ms, mirrors existing <code>SearchBar.js</code> debounce pattern
<code>shortcuts</code>	<code>Array<{combo,label,view}></code>	Registered actions, e.g. <code>Ctrl+K</code> open, <code>G then D</code> go-to-dashboard
<code>open</code> / <code>onClose</code>	bool / fn	Palette is a <code>Modal</code> -family overlay (composes existing <code>Modal.js</code> backdrop/escape logic) rather than a new overlay primitive
<code>results</code>	<code>Array<{group,items}></code>	Grouped: Screens, Clients, Parts, Invoices — sourced from <code>api.get</code> debounced calls per group

States: collapsed bar (icon + "Search or Ctrl+K" placeholder) · open palette (centered, 640px, arrow-key row navigation, Enter to activate) · loading (3-row `LoadingSkeleton variant="rows"` per group) · empty ("No matches for 'x'" + suggested screens) · error (inline `StatusPill tone="danger"` banner, existing `ApiError.message` surfaced verbatim).

KPITILE & KPIBAND

PROP (KPITILE)	TYPE	NOTES
<code>label</code> / <code>value</code>	string	<code>value</code> pre-formatted by caller via <code>MoneyText</code> or <code>ckNum</code> -style formatter, tile does not format
<code>delta</code>	<code>{text,dir}</code>	Same shape as current <code>ckPct/ckDelta</code> helpers in <code>dashboard-view.js</code> — reused, not reinvented
<code>icon</code>	string	Key into existing <code>CockpitIcons.js</code> registry (kept — icon set is design-agnostic)
<code>sparkline</code>	number[]	Inline SVG polyline, ≤7 points, no chart library
<code>tone</code>	success/danger/info/neutral/orange	Maps to token accent for icon chip + delta color
<code>onClick</code>	fn	Navigates, same pattern as current clickable <code>ck-kpi-card</code>

`KpiBand` is a thin wrapper: `{tiles: KpiTile[], columns: 4|6|8}` , renders a responsive CSS grid (12-col, tiles span 3 or 2 cols) — see Section 8 for breakpoints.

ACTIONQUEUECARD & INSIGHTPANEL

- `ActionQueueCard{tone,label,title,detail,value,onOpen}` — 1:1 replacement for the existing `buildActionQueue()` task shape already computed in `dashboard-view.js` (danger/warning/neutral/success tones map directly to Ignition danger/amber/info/success tokens).
- `InsightPanel{title,subtitle,children,footer}` — generic elevated card shell (replaces ad-hoc `ck-panel` divs); every dashboard panel (Inventory Snapshot, Top Categories, Sales Overview, Recent Activity) becomes one `InsightPanel` instance with different children.

TRENDCHART

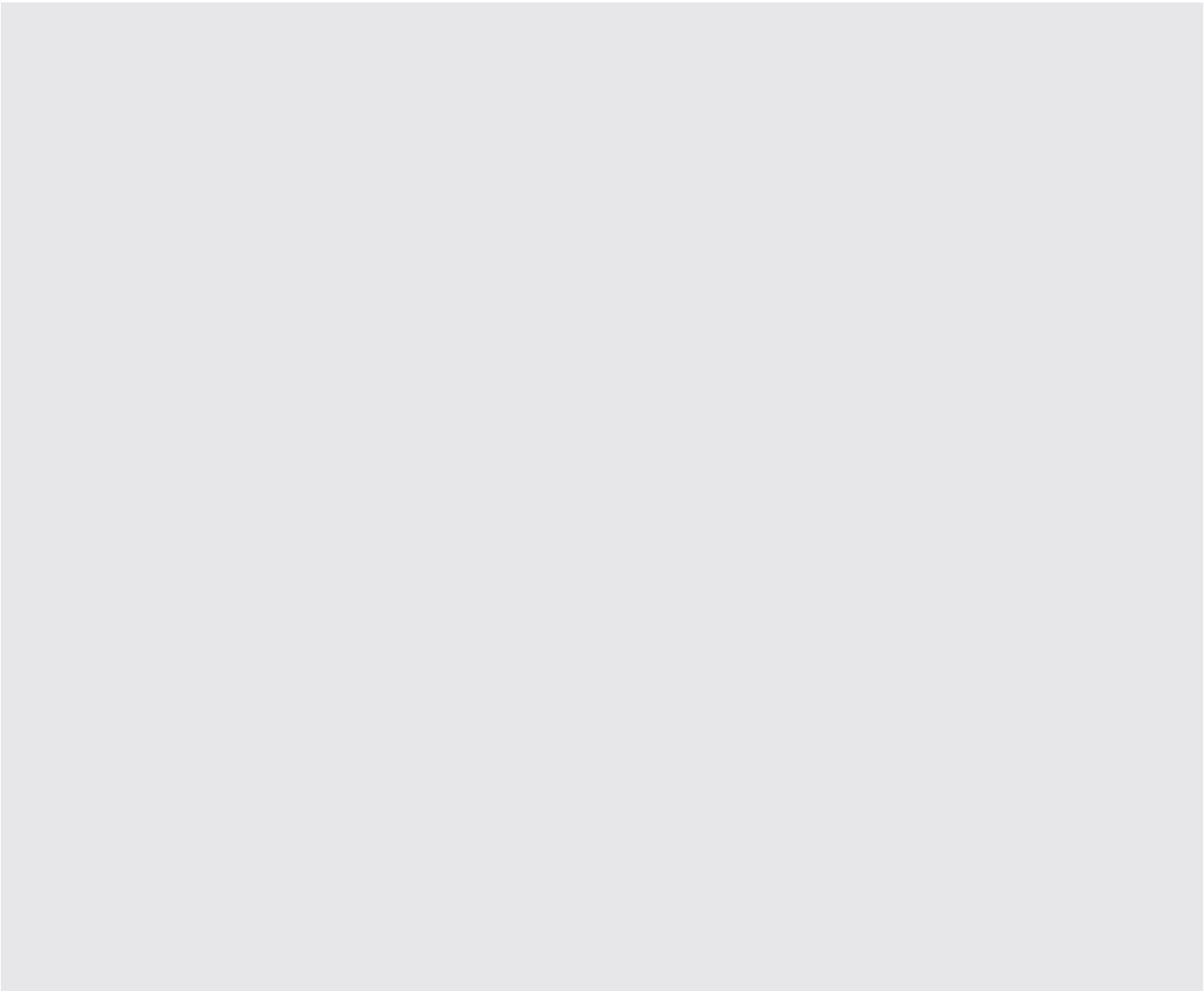
PROP	TYPE	NOTES
<code>series</code>	<code>Array<{label,value}></code>	Same shape as current <code>weekly</code> array + <code>ckLinePath()</code> helper — logic ported, not rewritten

<code>variant</code>	area/line/bar	All inline SVG, reuses existing gradient-fill technique from <code>dashboard-view.js</code> (<code>ckArea</code> / <code>ckStroke</code> gradients)
<code>height</code>	number	px; default 168 to match current chart
<code>formatValue</code>	fn	Injected formatter (keeps chart currency-agnostic per multi-tenant <code>displayCurrencyContext</code>)

States: default · loading (skeleton bars, animated opacity pulse, no layout shift once data arrives — fixed height reserved) · empty ("No sales recorded this period") · single-point (renders a flat reference line, avoids NaN from division by zero — bug present in current `ckLinePath` mitigated).

DATAGRID (SORTABLE / VIRTUALIZED)

PROP	TYPE	NOTES
<code>columns</code> / <code>rows</code> / <code>getRowKey</code>	same as <code>DataTable</code>	Backward-compatible superset — <code>DataGrid</code> wraps <code>DataTable</code> 's sort logic verbatim, adds virtualization
<code>virtualize</code>	bool	Auto-enabled when <code>rows.length > 150</code> (see Section 9 threshold)
<code>rowHeight</code>	number	Fixed row height required for windowing math (44px default)
<code>renderRowActions</code> / <code>renderExpanded</code>	fn	Identical contract to current <code>DataTable</code> — no breaking change for callers migrating later
<code>stickyColumns</code>	number	New: freezes first N columns horizontally (needed for wide invoice/payment grids)



Client Workspace Components

CLIENTRAIL

PROP	TYPE	NOTES
<code>clients</code>	<code>Array<{id,name,balance,riskTone,lastActivity}></code>	Merged customer+supplier list, replaces the two-column split in <code>contacts-view.js</code>
<code>selectedId</code>	<code>string</code>	Drives <code>ClientHeader</code> + all tabs
<code>onSelect(id)</code>	<code>fn</code>	
<code>onSearch(query)</code>	<code>fn</code>	Local filter first (instant), falls back to <code>api.get('/api/customers?search=')</code> for full-catalog search past 100 loaded rows
<code>filterKind</code>	<code>all/customer/supplier</code>	Segmented control at rail top

States: default row (name, muted balance) · row with overdue balance (danger `MoneyText` , small red dot) · selected (orange left bar + elevated bg) · loading (8× skeleton rows) · empty ("No clients match 'x'" + Clear filter action) · error (inline retry banner using existing `ApiError.message`).

CLIENTHEADER

PROP	TYPE	NOTES
<code>client</code>	<code>object</code>	name, phone, email, type, createdAt
<code>balance</code>	<code>{total,currency}</code>	Rendered via <code>MoneyText</code>
<code>riskTone</code>	<code>success/warning/danger</code>	Derived: danger if <code>over90Days > 0</code> , warning if any 1–90 bucket > 0 — logic ported from <code>customer-aging-view.js</code> <code>agingClass()</code>
<code>quickActions</code>	<code>Array<{label,icon,onClick}></code>	New Sale, Send Reminder (reuses <code>CommunicationPayloadFactory.accountBalanceReminder</code>), New Quote, Call/WhatsApp

AGINGBAR

PROP	TYPE	NOTES
<code>buckets</code>	<code>{current,d1to30,d31to60,d61to90,over90}</code>	Direct field-mapping from existing <code>/api/customers/aging</code> row shape — no API change needed
<code>total</code>	<code>number</code>	
<code>onBucketClick(bucket)</code>	<code>fn</code>	Filters the invoices tab to that bucket's open invoices

Segmented horizontal bar, width proportional to each bucket's share of `total` ; buckets beyond 60 days render in amber/danger tokens even at small proportions (accessibility: never rely on size alone — see Section 11).

ACTIVITYTIMELINE

PROP	TYPE	NOTES
<code>events</code>	<code>Array<{id,kind,title,detail,timestamp,actor}></code>	<code>kind</code> ∈ invoice/payment/quote/message/note/vehicle — merges <code>/api/communications</code> , <code>/api/sales</code> , <code>/api/customerPayments</code> per client, client-side sorted by timestamp desc
<code>groupByDay</code>	<code>bool</code>	Default true — sticky day-divider labels
<code>onLoadMore</code>	<code>fn</code>	Cursor pagination, 25/page

TABSTRIP & QUICKACTIONDOCK

- `TabStrip{tabs:[{key,label,count?}], activeKey, onChange}` — drives the 7 Client Workspace regions: Overview, Aging, Invoices, Payments, Quotes, Part Requests, Vehicles & Loyalty. Underline-style, keyboard arrow navigation (Section 9).
- `QuickActionDock{actions}` — sticky bottom-right dock on mobile/tablet (replaces desktop-only header actions when ClientHeader collapses, see Section 8).

DRAWER, STATUSPILL, MONEYTEXT

COMPONENT	CHANGE FROM EXISTING KIT
<code>Drawer</code>	Reused as-is from <code>js/components/ui/Drawer.js</code> — already supports <code>side/size/footer</code> , restyled purely via token CSS override, no API change. Used for invoice/payment/quote detail edit flows launched from the workspace.

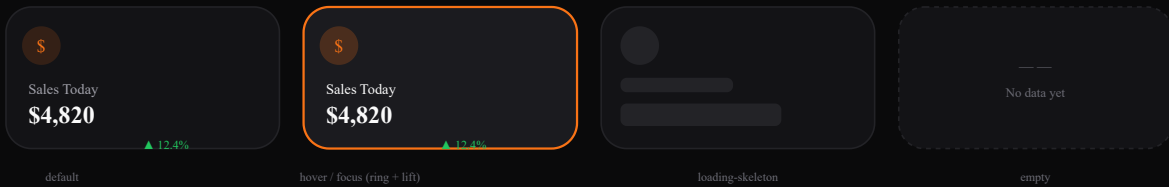
StatusPill Reused as-is from `js/components/ui/StatusPill.js` — its tone map (success/warning/danger/info/neutral) already matches Ignition's 5-tone system 1:1. Only the CSS custom properties it renders against change.

MoneyText **New.** `{value,currency,size,negativeTone}` — renders tabular-numeral `<span>` via `font-variant-numeric: tabular-nums + --ig-font-mono` ; negative values get `--ig-danger` color and a leading minus, never parentheses (matches existing `danger-text` convention in `formatters.js` consumers).

Rendered Specimens — KpiTile, AgingBar, ClientRail Row

Every Ignition component ships five required states: **default**, **hover/focus**, **loading-skeleton**, **empty**, **error**. The specimen sheet below is drawn to scale as it will render in the browser (not a mock) to lock exact spacing, radius, and color values before implementation.

KpiTile — state variants



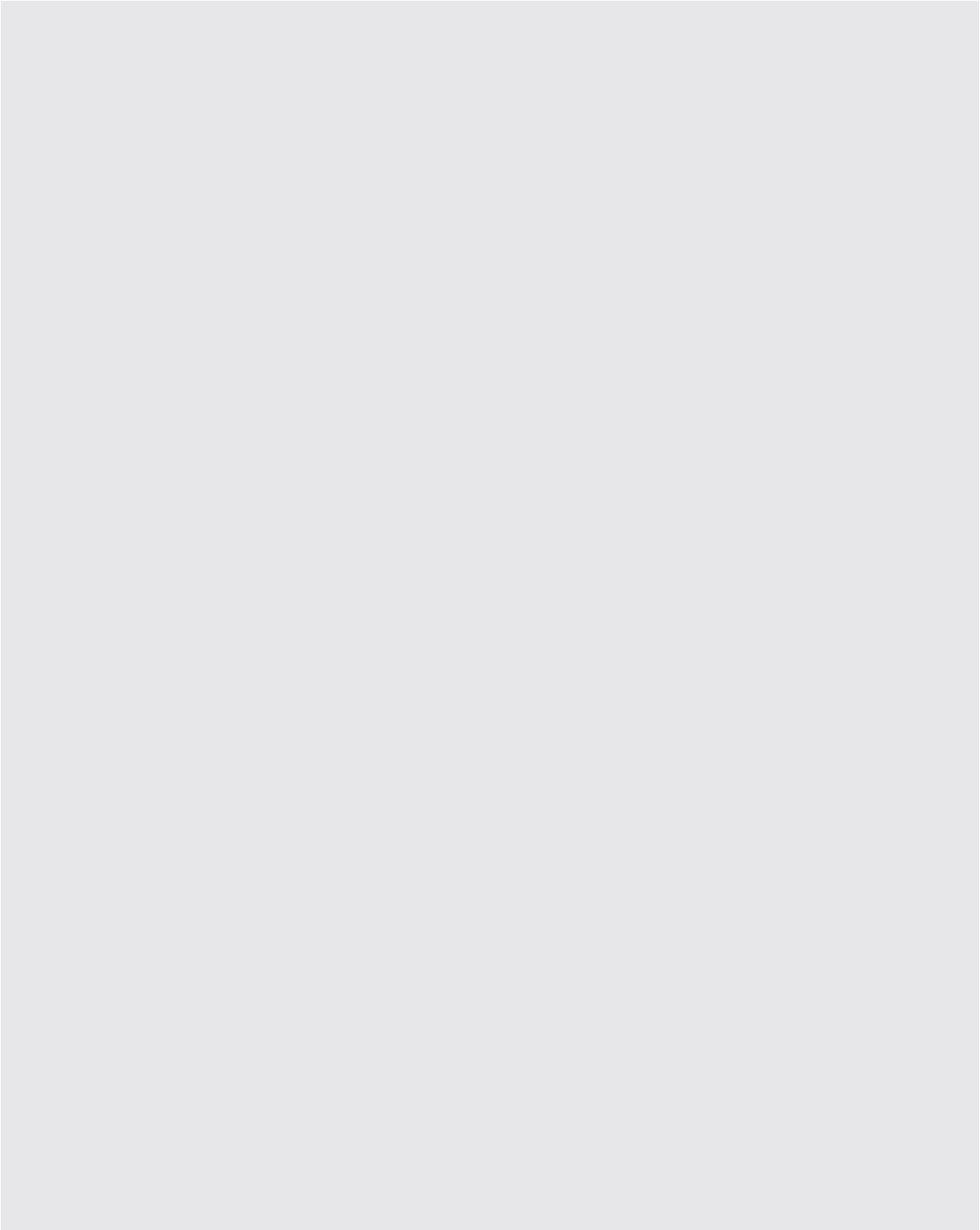
AgingBar — segmented (with error state)



ClientRail row — default vs selected vs overdue



All specimens drawn at implementation scale — 16px radius, 8px spacing rhythm, JetBrains Mono for every numeral.

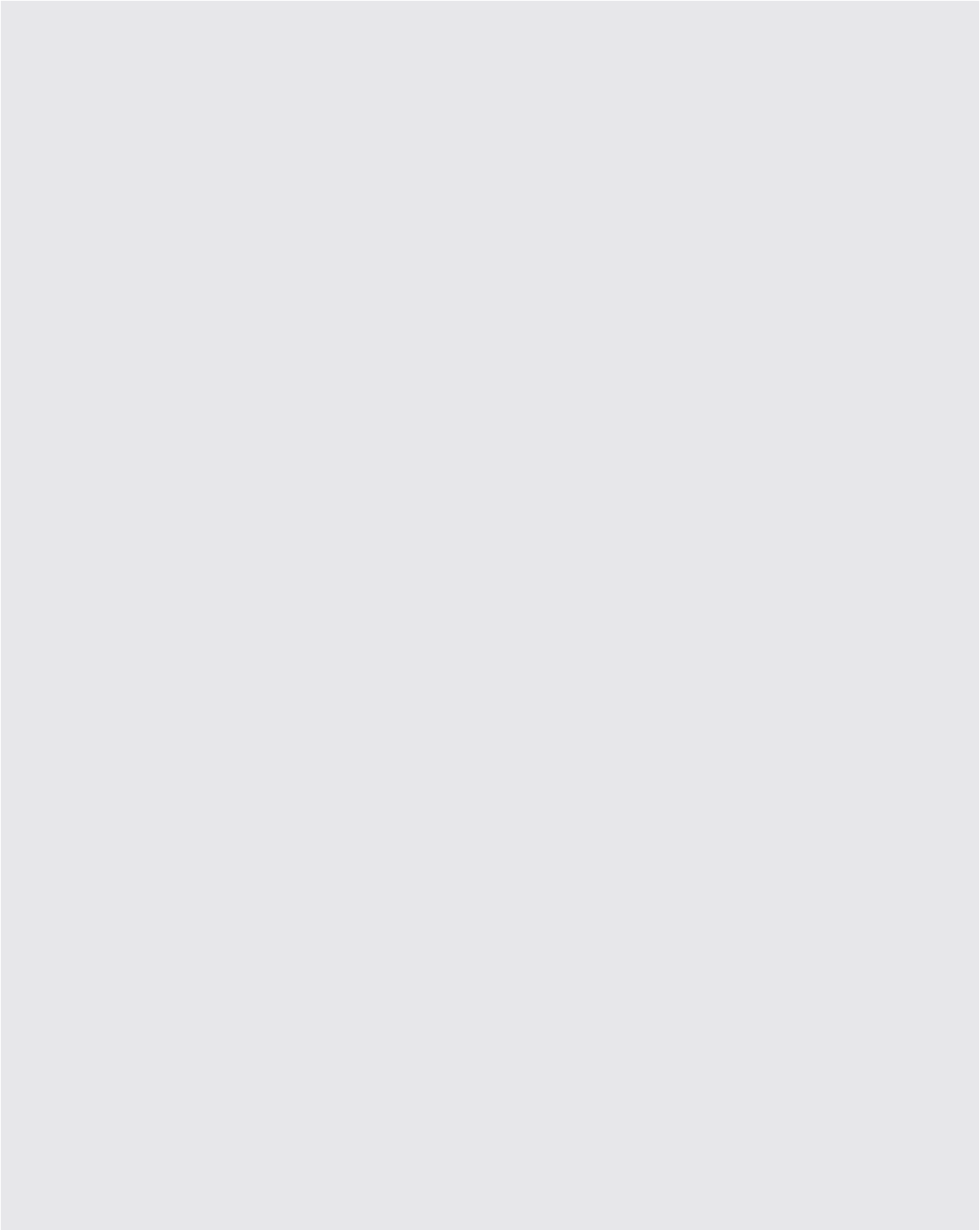


Dashboard Screen — Component Tree

Data flows top-down from a single `load()` effect in `dashboard-view.js` (unchanged pattern: `Promise.all` across `/api/owner-cockpit`, `/api/communications/recent`, `/api/appconstants`, `/api/currencies`, `/api/parts`, `/api/partrequests`, `/api/customers/aging`) into derived view-model objects, which are passed as props into Ignition components — no component below fetches its own data.



Fig. 2 — Dashboard component tree with data-prop provenance (Ignition version)



Client Workspace Screen — Component Tree

`client-workspace-view.js` replaces `contacts-view.js` + `customer-aging-view.js` + fragmented customer context in invoices/quotes/part-requests/loyalty. It owns one `selectedClientId` state; the tab body components lazy-load their own data slice keyed off that id, mirroring the existing per-view `load()` / `useCallback` pattern.

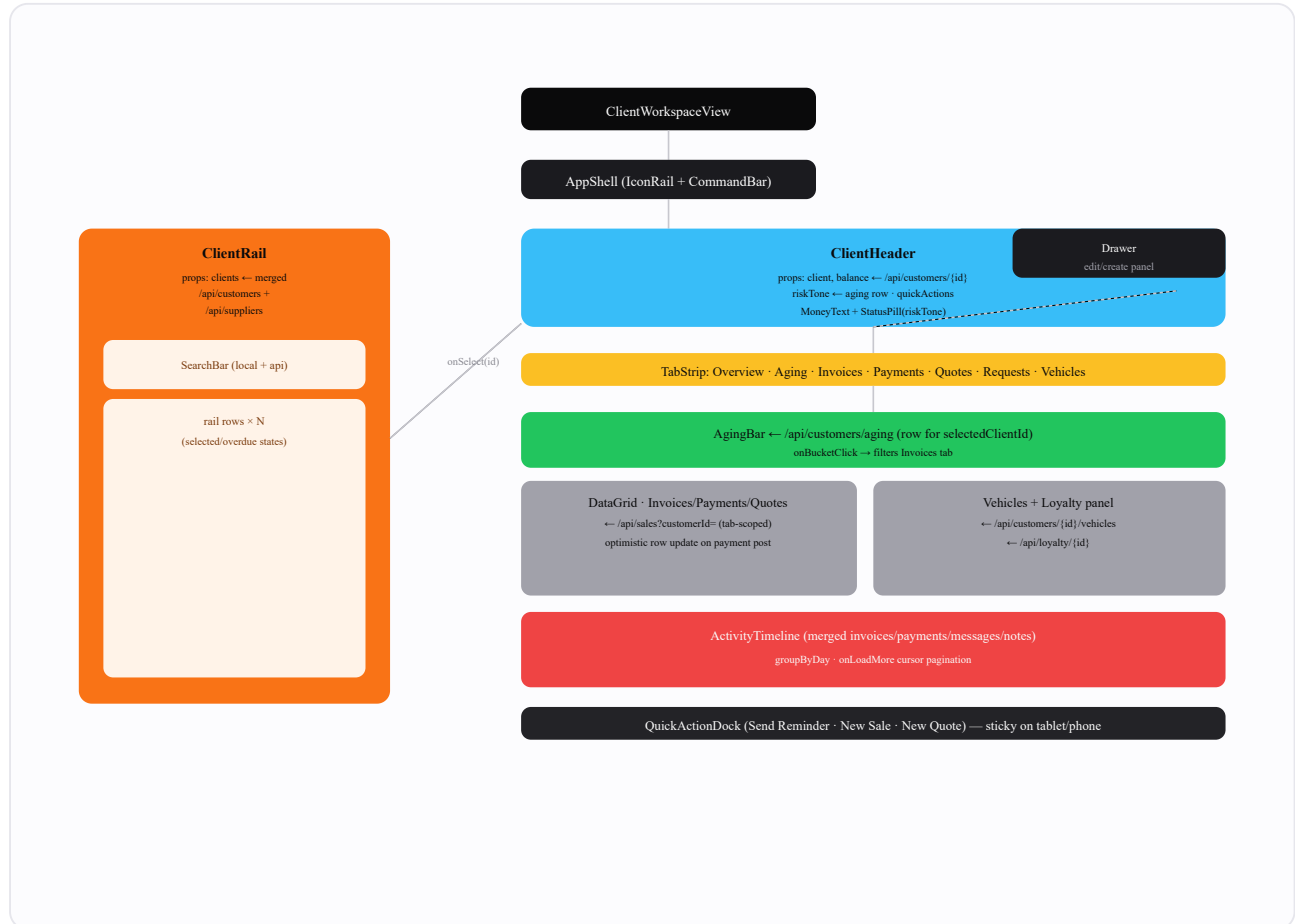
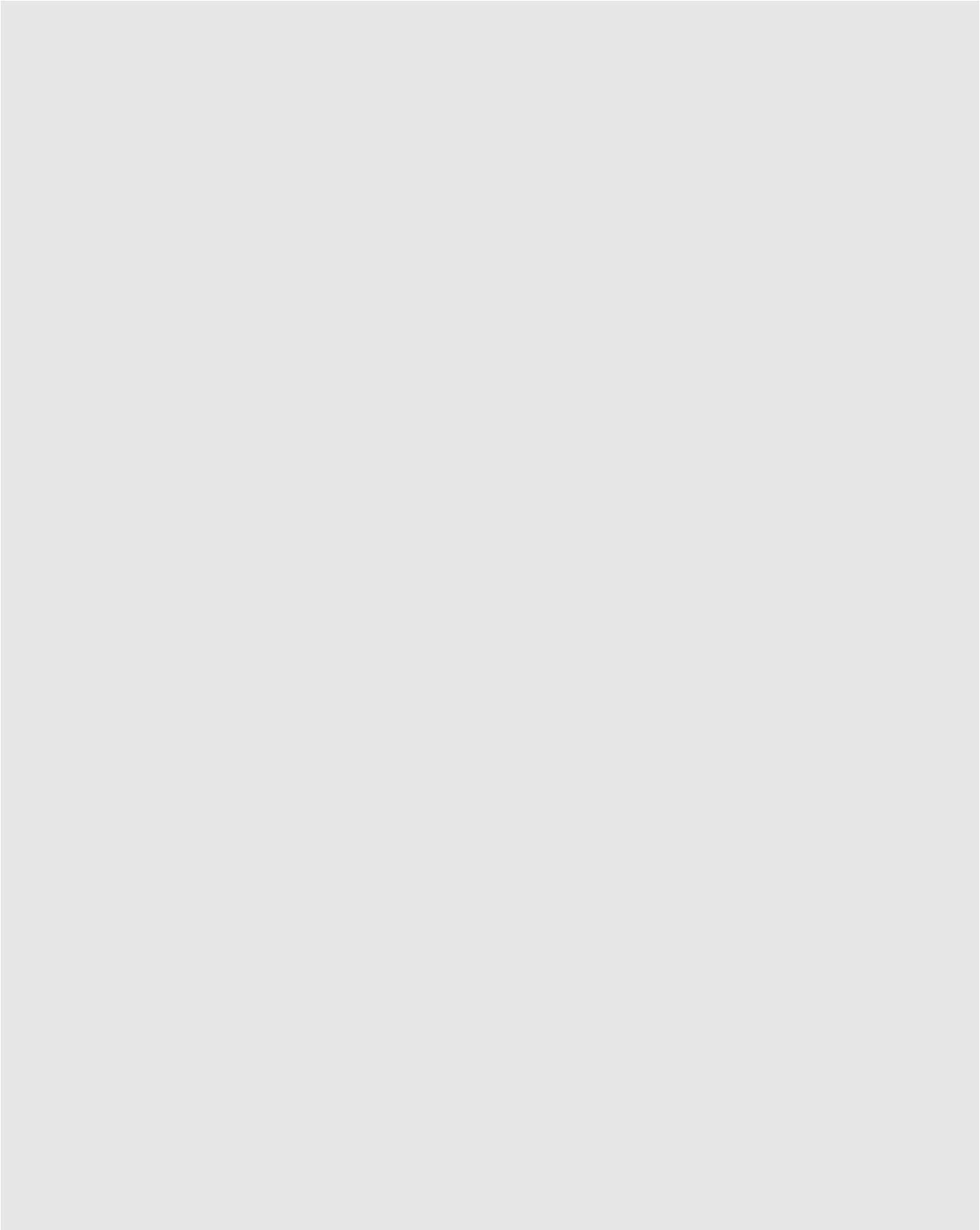


Fig. 3 — Client Workspace component tree; ClientRail selection drives every region to the right



Breakpoints & Reflow

TOKEN	RANGE	GRID	ICONRAIL	KPIBAND	CLIENTRAIL
--ig-bp-desktop	≥1280px	12 col, 24px gutter	72px fixed, flyout on hover	4 cols × 2 rows (8 tiles)	280px fixed left column
--ig-bp-laptop	1024–1279px	12 col, 20px gutter	72px fixed, flyout on hover	3 cols wrap	240px fixed
--ig-bp-tablet	768–1023px	8 col, 16px gutter	collapses to bottom tab bar (5 primary items) OR top overflow menu	2 cols, horizontally scroll-snap if >6 tiles	becomes a full-width collapsible sheet above ClientHeader, selected client shown as a compact chip
--ig-bp-phone	≤767px	4 col, 12px gutter	bottom tab bar, 5 items + "More" sheet	1 col, horizontal scroll-snap carousel	full-screen searchable list (Drawer), tapping a client pushes ClientHeader+Tabs as a new "page" (back button)

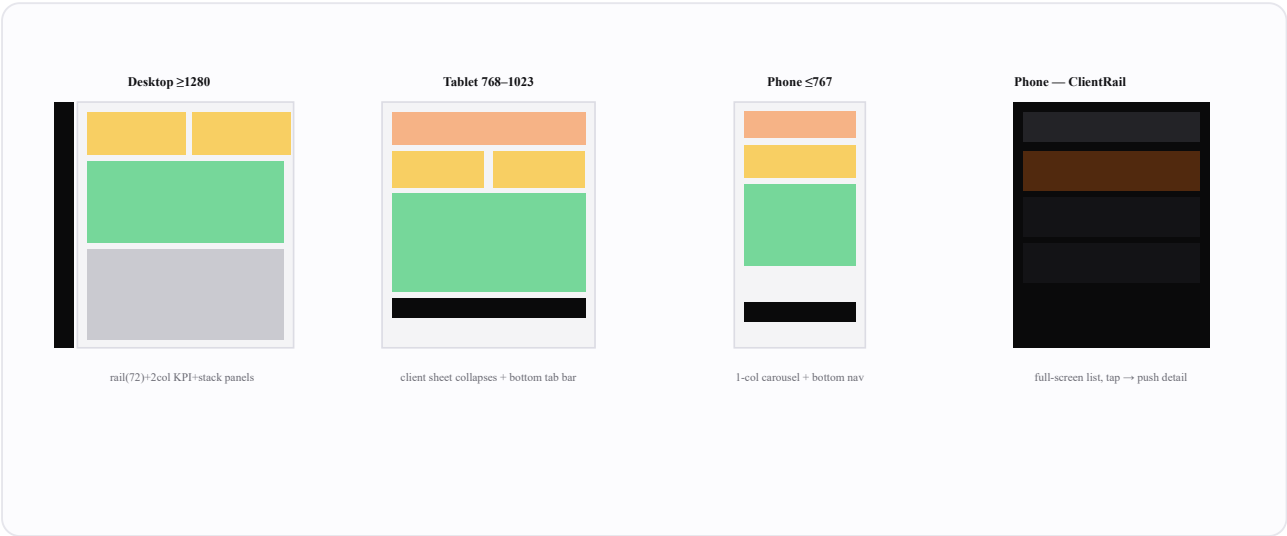
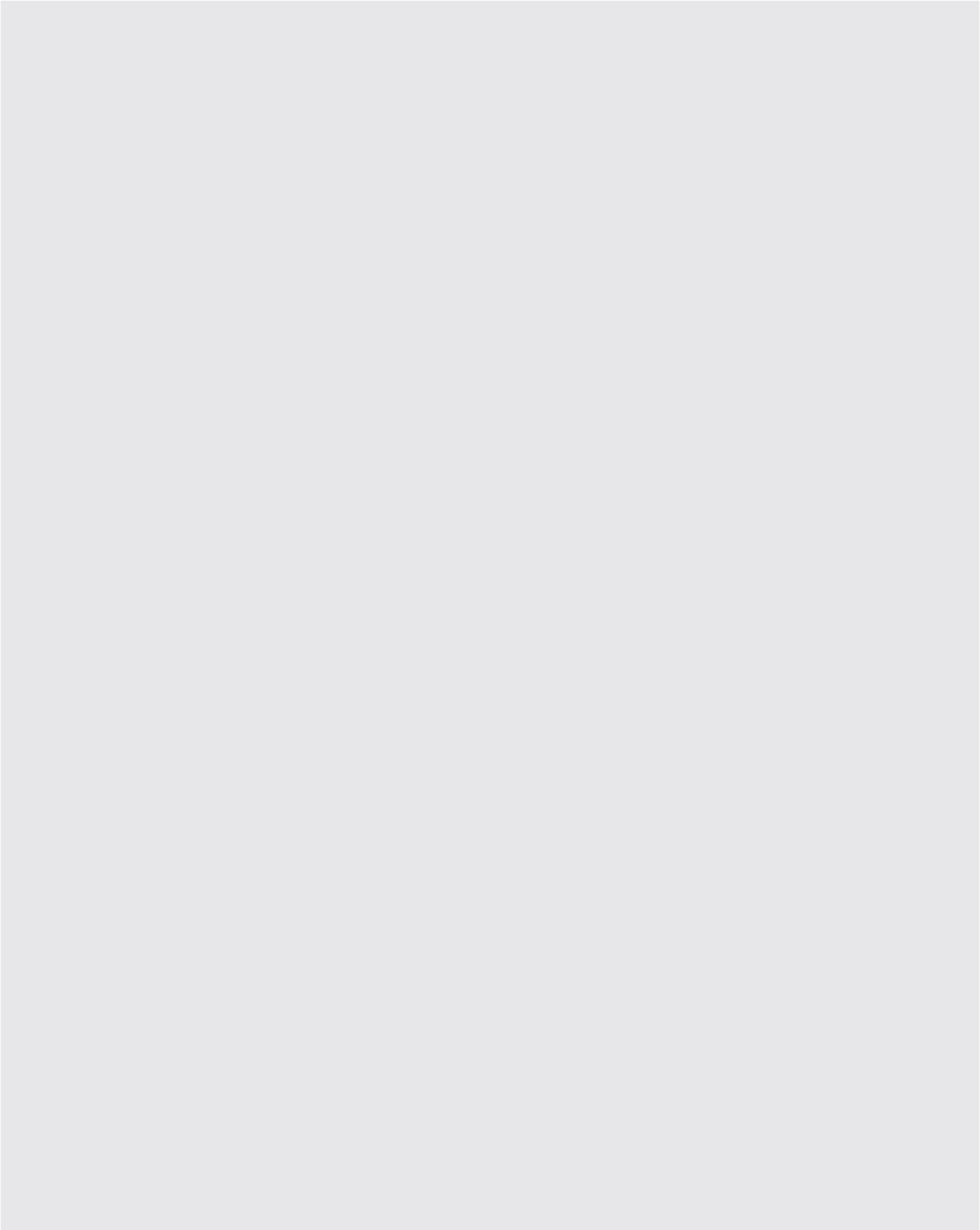


Fig. 4 — Reflow across the four breakpoints (schematic, drawn to relative scale)

TOUCH BEHAVIOR

- All interactive targets ≥44×44px on tablet/phone (IconRail bottom-tab icons, ClientRail rows, TabStrip tabs).
- DataGrid on phone drops to a card-list rendering (same row data, `columns` prop reduced to primary+secondary fields) rather than horizontal-scroll table — horizontal scroll only kept on tablet with sticky first column via `stickyColumns`.
- CommandPalette (Ctrl+K) has no phone keyboard-shortcut equivalent; phone shows a persistent search icon in the bottom tab bar instead.
- Swipe-back gesture (native browser back / History API) closes the pushed client-detail "page" on phone, mirroring the Drawer's Escape-to-close affordance on desktop.



Keyboard, Focus, Optimistic UI, Loading Choreography

KEYBOARD SHORTCUT MAP

COMBO	ACTION	SCOPE
Ctrl/Cmd + K	Open CommandPalette	Global (Ignition screens)
Esc	Close CommandPalette / Drawer / Modal (existing behavior in <code>Drawer.js</code> / <code>Modal.js</code> — reused verbatim)	Overlay-scoped
↑ / ↓	Move selection in CommandPalette results, ClientRail list	List-scoped
Enter	Activate selected palette result / navigate to focused client	List-scoped
← / →	Move active TabStrip tab (roving tabindex, ARIA <code>tablist</code> pattern)	TabStrip-scoped
G then D	Go to Dashboard	Global
G then C	Go to Client Workspace	Global
/	Focus ClientRail search when workspace is active	Client Workspace

FOCUS MANAGEMENT

- Opening CommandPalette/Drawer moves focus to the first focusable element inside (search input / first form field); closing returns focus to the trigger element (stored via `useRef` before open, matching the ref pattern already used across existing views).
- Focus trap inside open overlays: Tab/Shift+Tab cycle within the overlay only — implemented with a small focusable-elements query helper, no new dependency.
- Route change (IconRail select / ClientRail select) sets focus to the new region's `h1` /heading via a shared `useFocusOnChange(dep)` hook so screen readers announce navigation.
- `:focus-visible` only (not `:focus`) shows the 2px orange ring, so mouse clicks stay visually clean while keyboard nav stays fully legible.

OPTIMISTIC UI — PAYMENT & INVOICE ACTIONS

```
// pattern: apply locally, reconcile with server response, roll back on ApiError
async function recordPayment(invoiceId, amount) {
  // 1. optimistic patch — DataGrid row updates instantly
  setInvoices(prev => prev.map(inv =>
    inv.id === invoiceId
      ? { ...inv, paidAmount: inv.paidAmount + amount, status: "Pending sync" }
      : inv));

  try {
    const updated = await api.post(`/api/sales/${invoiceId}/payments`, { amount });
    // 2. reconcile with server truth
    setInvoices(prev => prev.map(inv => inv.id === invoiceId ? updated : inv));
    toast.success("Payment recorded.");
  } catch (error) {
    // 3. rollback + surface ApiError.message (existing ApiClient contract)
    setInvoices(prev => prev.map(inv => inv.id === invoiceId ? originalRow : inv));
    toast.error(error.message || "Payment failed — reverted.");
  }
}
```

Applies to: payment recording, quote status change, part-request status change, reminder send (button shows inline spinner but list item itself does not optimistically change, since delivery is asynchronous and best confirmed by the existing `reminderCustomerId`-style pending-state pattern already in `customer-aging-view.js`).

SKELETON LOADING CHOREOGRAPHY

- T+0ms: view mounts, all regions render `LoadingSkeleton` variants matching final layout shape (existing component, reused as-is — `variant="stats"` for KpiBand, `variant="rows"` for DataGrid/ClientRail, `variant="cards"` for InsightPanel).
- T+≤150ms: if data resolves before this threshold, skip the skeleton fade-in entirely to avoid flash-of-loading-state on fast connections.
- Data arrives: skeleton cross-fades to real content over 160ms (`--ig-dur`), region by region as each `Promise.all` branch resolves — KpiBand and ClientRail can populate before

TrendChart/ActivityTimeline finish.

4. Virtualized DataGrid threshold: rows > 150 auto-enables windowing (fixed 44px row height, ~12 rows rendered + overscan 4); below 150 rows renders fully, matching current `DataTable` behavior with zero risk of regression for the majority of screens (aging table today typically renders 20-60 rows).

Render Targets, Module Loading, Memoization, Chart Rendering

<1.2s DASHBOARD FIRST RENDER	<150ms SKELETON→CONTENT CROSSFADE	≤12 ES MODULE FILES/VIEW (INITIAL)	90+ LIGHTHOUSE PERF TARGET
---	---	--	--------------------------------------

MODULE LOADING STRATEGY (NO-BUILD RUNTIME)

- Every Ignition component file is a native ES module (`export function X(){}`), loaded by the browser's module graph via static `import` — no bundler means no code-splitting tooling, so the strategy is **discipline in the import graph**, not a build config: shell components (`IconRail` , `CommandBar`) import eagerly since every screen needs them; heavy, screen-specific pieces (`TrendChart` , `ActivityTimeline`) are only imported by the views that render them, so browser HTTP/2 module fetching naturally lazy-loads per-route.
- `<link rel="modulepreload">` added to `index.html` for the shell components (AppShell/IconRail/CommandBar/tokens) so they're warm before the router resolves the first screen — same technique already usable with zero new infra since these are static files served by the existing ASP.NET static-file host.
- All 4 CSS layer files are small (tokens ~2KB, primitives ~4KB, components ~14KB, per-view ~3KB) — total new CSS payload for the two Ignition screens stays under 25KB gzipped, versus continuing to grow the single 17,400-line file.

MEMOIZATION RULES

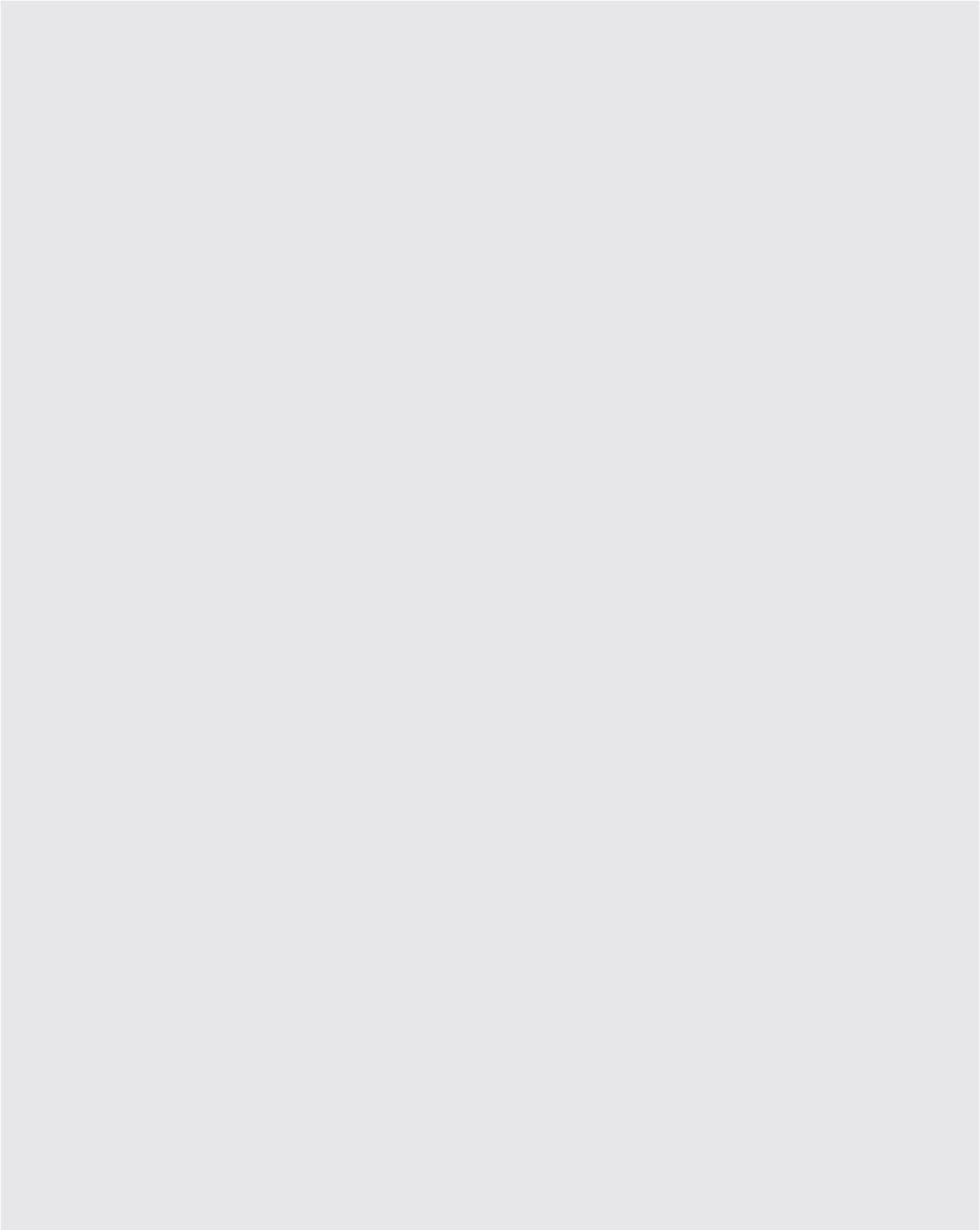
- Derived view-model computations (aging totals, KPI deltas, category rollups) wrapped in `useMemo` keyed on the raw API arrays — ports the existing pattern already present in `customer-aging-view.js` (`totals` , `over30` , `over90` via `useMemo`) rather than introducing a new pattern.
- List item renderers (ClientRail rows, DataGrid rows) are plain functions, not wrapped in `React.memo` , unless profiling on the real dataset shows re-render cost — the runtime exposes `useMemo/useCallback` only (see `react-runtime.js`), no `memo` / `lazy` re-export today; adding that export is a one-line, low-risk addition to `react-runtime.js` if profiling later requires it.
- Event handlers passed to list rows are created once per row via a stable factory keyed by id (`onSelect(row.id)`) curried at row-render time is acceptable at current data volumes — client lists are typically <500 rows per tenant).

CHART RENDERING APPROACH

All charts (TrendChart, KpiTile sparkline, AgingBar) are hand-built inline SVG using the same technique already proven in `dashboard-view.js`'s `ckLinePath()` / `ckSpark()` helpers — no charting library dependency, no new bundle weight, full styling control via CSS custom properties (stroke/fill reference `--ig-orange` etc. so dark/light theme switching repaints charts for free without JS re-computation).

TARGET LIGHTHOUSE METRICS (DASHBOARD + CLIENT WORKSPACE, DESKTOP CONFIG)

METRIC	TARGET	RATIONALE
First Contentful Paint	< 1.0s	Static shell (rail/topbar/skeletons) has no data dependency
Largest Contentful Paint	< 1.8s	KpiBand is the LCP element; resolves as soon as <code>/api/owner-cockpit</code> returns
Total Blocking Time	< 150ms	No heavy synchronous computation on main thread; aging/category rollups are O(n) over small arrays
Cumulative Layout Shift	< 0.05	Skeletons reserve exact final height (fixed KpiTile height, fixed 168px chart height) — no shift on data arrival
Lighthouse Performance score	≥ 90	No render-blocking third-party scripts introduced; Google Fonts loaded with <code>font-display: swap</code>



ARIA Landmarks, Focus-Visible, Contrast

ARIA ROLES PER COMPONENT

COMPONENT	ROLE / ATTRIBUTES
AppShell regions	<code><nav aria-label="Primary"></code> (IconRail), <code><header role="banner"></code> (CommandBar/Topbar), <code><main aria-label="{screen title}"></code> (content)
IconRail	Each item: <code><button aria-current="page"></code> when active; <code>title</code> + visually-hidden text label for the flyout so screen readers get the label even when collapsed
CommandPalette	<code>role="dialog" aria-modal="true" aria-label="Command palette"</code> ; results list is <code>role="listbox"</code> with <code>role="option"</code> children and <code>aria-activedescendant</code> tracking arrow-key selection
KpiTile / KpiBand	KpiBand is <code>role="list"</code> , each tile <code>role="listitem"</code> ; delta arrows include <code>aria-label="up 12.4 percent versus yesterday"</code> (icon alone is <code>aria-hidden</code>)
TabStrip	Standard ARIA tabs pattern: <code>role="tablist"</code> / <code>role="tab" aria-selected</code> <code>aria-controls</code> / panel <code>role="tabpanel" aria-labelledby</code> , roving <code>tabindex</code>
DataGrid	Native <code><table></code> retained (matches existing <code>DataTable</code> semantics) with <code><caption class="sr-only"></code> describing the dataset; sortable headers get <code>aria-sort</code>
Drawer / CommandPalette overlay	<code>aria-modal="true"</code> , focus trap, <code>Esc</code> to close, backdrop <code>aria-hidden="true"</code> on background content while open
ClientRail	<code>role="listbox"</code> , rows <code>role="option" aria-selected</code> ; search input has an associated <code><label class="sr-only"></code>
StatusPill / MoneyText	Never color-only: StatusPill always renders an icon + text label (already true in current implementation); MoneyText negative values get a leading minus sign, not color alone

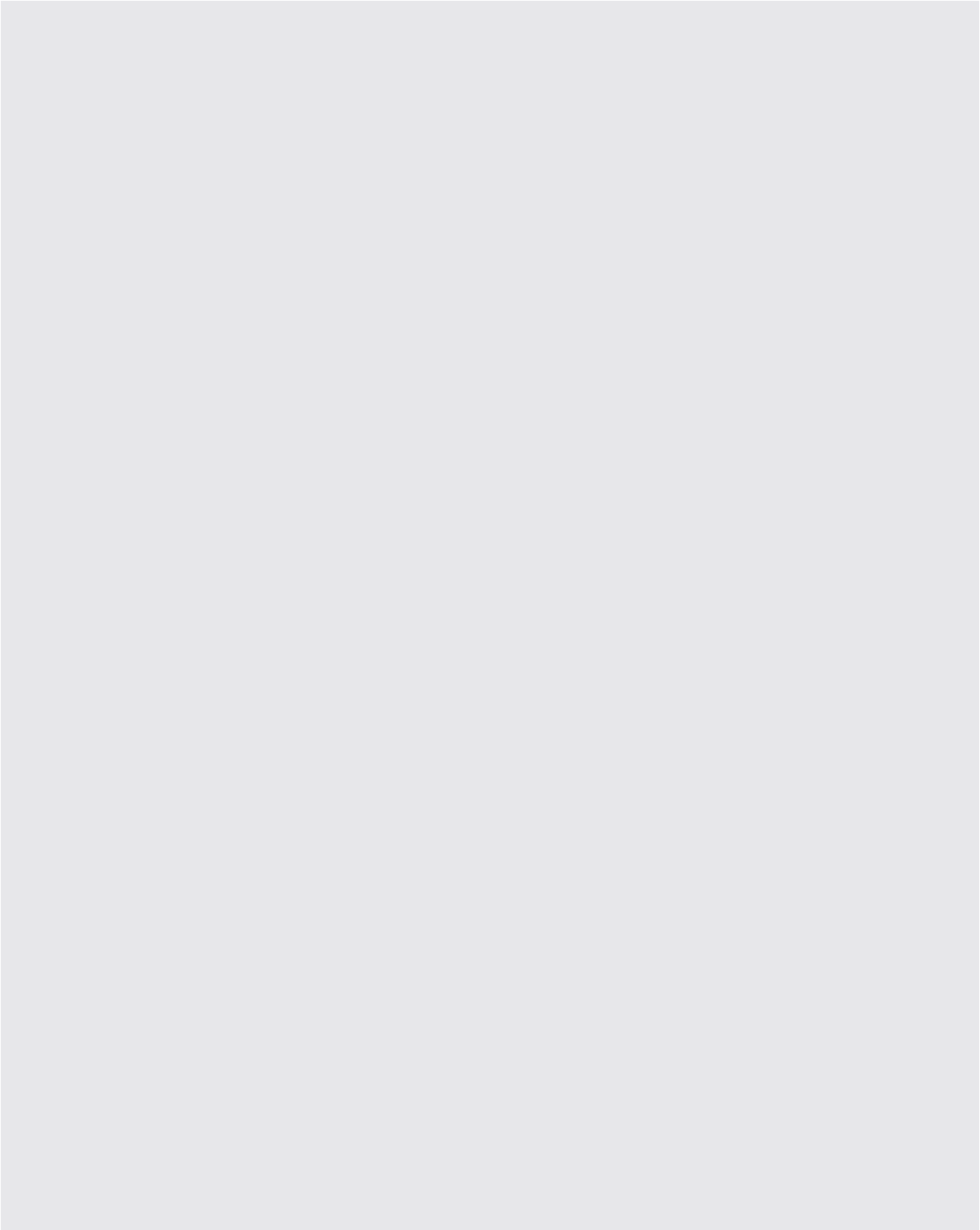
FOCUS-VISIBLE TREATMENT

```
.ignition-root :focus-visible {
  outline: 2px solid var(--ig-orange);
  outline-offset: 2px;
  border-radius: 6px;
}
/* mouse clicks never show the ring - :focus-visible is UA-computed */
.ignition-root button:focus:not(:focus-visible) { outline: none; }
```

CONTRAST-CHECKED TOKEN PAIRS (WCAG 2.1 AA, 4.5:1 BODY / 3:1 LARGE TEXT)

FOREGROUND	BACKGROUND	RATIO	USAGE	RESULT
#fafafa	#0a0a0b	19.4:1	Primary text on carbon	Pass AAA
#a1a1aa	#0a0a0b	7.8:1	Secondary text on carbon	Pass AAA
#a1a1aa	#131316	7.1:1	Secondary text on surface	Pass AAA
#0a0a0b	#f97316	7.6:1	Text on orange CTA fill	Pass AAA
#fafafa	#ef4444	4.6:1	Text on danger fill (buttons only, not body text)	Pass AA (large/bold)
#22c55e	#131316	6.4:1	Success delta text	Pass AAA
#131316	#ffffff	18.1:1	Light-mode primary text	Pass AAA
#55555f	#ffffff	6.7:1	Light-mode secondary text	Pass AAA

All pairs actually used for body copy or button labels clear 4.5:1; the one 4.6:1 pair (white-on-danger) is restricted to bold 13px+ button labels, which only require 3:1 under AA — comfortable margin retained regardless.



File-by-File Map, Ordered in Shippable Phases

PHASE 0 TOKEN FOUNDATION (NO VISUAL CHANGE YET, ZERO SCREEN-FACING RISK)

FILE	ACTION
<code>wwwroot/css/ignition/tokens.css</code>	Create — color/space/type/radius/shadow/motion custom properties, dark + <code>[data-ig-theme="light"]</code>
<code>wwwroot/css/ignition/primitives.css</code>	Create — reset, 12-col grid, stack/cluster utilities, focus-visible ring, sr-only, all scoped under <code>.ignition-root</code>
<code>wwwroot/index.html</code>	Edit — add two <code><link></code> tags after existing <code>styles.css</code> / <code>cockpit.css</code> links, plus Google Fonts <code>@import</code> fallback stack

PHASE 1 COMPONENT LIBRARY — SHELL & DATA PRIMITIVES

FILE	ACTION
<code>wwwroot/css/ignition/components.css</code>	Create — <code>.ig-*</code> rules for every component in Section 3–4
<code>wwwroot/js/components/ignition/IconRail.js</code>	Create
<code>wwwroot/js/components/ignition/CommandBar.js</code>	Create
<code>wwwroot/js/components/ignition/CommandPalette.js</code>	Create (composes existing <code>Modal.js</code> escape/backdrop logic)
<code>wwwroot/js/components/ignition/KpiTile.js</code>	Create
<code>wwwroot/js/components/ignition/KpiBand.js</code>	Create
<code>wwwroot/js/components/ignition/ActionQueueCard.js</code>	Create
<code>wwwroot/js/components/ignition/InsightPanel.js</code>	Create
<code>wwwroot/js/components/ignition/TrendChart.js</code>	Create (ports <code>ckLinePath</code> / <code>ckSpark</code> math from <code>dashboard-view.js</code>)
<code>wwwroot/js/components/ignition/DataGrid.js</code>	Create (wraps <code>DataTable.js</code> sort logic, adds virtualization + sticky columns)
<code>wwwroot/js/components/ignition/MoneyText.js</code>	Create
<code>wwwroot/js/components/ignition/QuickActionDock.js</code>	Create

PHASE 2 DASHBOARD V2 (FIRST SHIPPED SCREEN)

FILE	ACTION
<code>wwwroot/css/ignition/views.dashboard.css</code>	Create — region layout only, all visual tokens inherited
<code>wwwroot/js/views/dashboard-view.js</code>	Replace — same <code>load()</code> /data-fetch logic and <code>buildActionQueue()</code> / <code>marginWatchPart()</code> helpers retained verbatim; JSX-equivalent markup rebuilt with Ignition components; root wrapped in <code>.ignition-root</code>
<code>wwwroot/js/views/screen-registry.js</code>	Edit — no key change, same <code>DashboardView</code> import path, zero-impact swap

PHASE 3 CLIENT WORKSPACE (NEW SCREEN, REPLACES FRAGMENTED VIEWS)

FILE	ACTION
<code>wwwroot/css/ignition/views.client-workspace.css</code>	Create
<code>wwwroot/js/components/ignition/ClientRail.js</code>	Create
<code>wwwroot/js/components/ignition/ClientHeader.js</code>	Create
<code>wwwroot/js/components/ignition/AgingBar.js</code>	Create (ports bucket math from <code>customer-aging-view.js</code>)
<code>wwwroot/js/components/ignition/ActivityTimeline.js</code>	Create
<code>wwwroot/js/components/ignition/TabStrip.js</code>	Create
<code>wwwroot/js/views/client-workspace-view.js</code>	Create — merges data calls currently split across <code>contacts-view.js</code> and <code>customer-aging-view.js</code> , reuses <code>CommunicationPayloadFactory.accountBalanceReminder</code> for the reminder action
<code>wwwroot/js/views/screen-registry.js</code>	Edit — add <code>{ key: "client-workspace", label: "Clients", component: ClientWorkspaceView }</code> ; leave <code>contacts</code> / <code>customer-aging</code> keys registered but remove from Sidebar's curated groups pending Ralph's sign-off to fully retire
<code>wwwroot/js/components/ui/Sidebar.js</code>	Edit — add "Clients" entry to the sidebar groups pointing at <code>client-workspace</code>

Files explicitly **not** touched in this plan: `styles.css` , `css/cockpit.css` , `contacts-view.js` , `customer-aging-view.js` , all other 66 view files, `api-client.js` , `react-runtime.js` — deletion of the superseded contact/aging views requires separate explicit approval per the Safety Rules before any file removal.